Navier–Stokes: POD-Galerkin vs PODNN

Lorenzo De Toffol
Model Order Reduction

October 20, 2026

Abstract

We solve the parametrized stationary Navier–Stokes problem on the unit square with a finite element full order model (FOM), and we build two reduced order models (ROMs) on top of it: a POD-Galerkin method and a POD neural network (PODNN). The two approaches are compared in terms of accuracy with respect to the FOM and computational cost on 50 unseen parameters. A large part of the work concerns the numerical conditioning of the reduced saddle-point system and the efficiency of the offline/online split. With the final implementation the POD-Galerkin ROM reaches mean relative errors of about $1{\cdot}10^{-4}$ (velocity) and $3{\cdot}10^{-4}$ (pressure) at a speed-up of about three orders of magnitude, while the PODNN is as fast but about two orders of magnitude less accurate.

1 Introduction

We consider the parametrized problem on $\Omega = (0,1)^2$: given $\boldsymbol{\mu} = (\mu_0, \mu_1) \in \mathcal{P} = [0.1, 10] \times [1, 3]$, find $(\mathbf{u}, p)$ such that

$$\begin{cases} -\mu_0 \, \Delta \mathbf{u} + (\nabla \mathbf{u})\mathbf{u} + \nabla p = \mathbf{f}(\mathbf{x}; \mu_1) & \text{in } \Omega, \\ \nabla \cdot \mathbf{u} = 0 & \text{in } \Omega, \\ \mathbf{u} = \mathbf{0} & \text{on } \partial\Omega, \end{cases} \tag{1}$$

with the pressure fixed by $p = 0$ at the vertex $(0, 0)$, and the source term $\mathbf{f}$ given. Here μ_0 plays is a viscosity (the Reynolds number scales like $1/\mu_0$) and μ_1 controls the spatial frequency of the forcing.

2 FOM

2.1 Finite element discretisation

The weak form of (1) is: find $(\mathbf{u}, p) \in V \times Q$ such that

$$\mu_0 \, a(\mathbf{u}, \mathbf{v}) + c(\mathbf{u}; \mathbf{u}, \mathbf{v}) - b(\mathbf{v}, p) = (\mathbf{f}, \mathbf{v}), \qquad b(\mathbf{u}, q) = 0 \qquad \forall (\mathbf{v}, q) \in V \times Q, \tag{2}$$

with $a(\mathbf{u}, \mathbf{v}) = (\nabla \mathbf{u}, \nabla \mathbf{v})$, $c(\mathbf{w}; \mathbf{u}, \mathbf{v}) = ((\nabla \mathbf{u})\mathbf{w}, \mathbf{v})$, $b(\mathbf{v}, q) = (q, \nabla \cdot \mathbf{v})$, $V = (H^1_0(\Omega))^2$ and $Q = L^2(\Omega)$ with the pressure fixed at the origin. We use the inf-sup stable Taylor–Hood pair $\mathbb{P}_2$–$\mathbb{P}_1$ on a triangular mesh with 830 vertices. The discrete problem has 6761 degrees of freedom (2×2966 for the velocity, 829 free plus one fixed for the pressure), 469 of which are constrained by the boundary conditions and the pressure gauge.

We denote by A the velocity stiffness matrix, B the discrete divergence, M_p the pressure mass matrix, $N(\mathbf{u})$ the convective vector and $C(\mathbf{u}) = \partial N / \partial \mathbf{u}$ its Jacobian. The nonlinear system is solved by Newton's method starting in zero:

$$\begin{bmatrix} \mu_0 A + C(\mathbf{u}_k) & -B^\top \\ -B & 0 \end{bmatrix} \begin{bmatrix} \delta \mathbf{u} \\ \delta p \end{bmatrix} = \begin{bmatrix} \mathbf{f} - \mu_0 A \mathbf{u}_k - N(\mathbf{u}_k) + B^\top p_k \\ B \mathbf{u}_k \end{bmatrix}, \tag{3}$$

(the continuity equation is multiplied by -1 so that the matrix is symmetric when $C = 0$). The iteration stops when the update is below a relative tolerance of 10^{-6}: it needs about three iterations, and one FOM solve takes about 0.6 s on Colab.

2.2 Snapshot dataset

We generate 1000 training and 50 test snapshots with parameters drawn uniformly at random in $\mathcal{P}$ (with a fixed seeds, so that the experiments are reproducible). Each snapshot stores the full vector of velocity and pressure degrees of freedom. The test parameters are never used to build the ROMs.

3 Reduced order models

3.1 POD bases

Let X be the H^1_0 inner product matrix on the velocity space (here $X = A$) and M_p the L^2 inner product matrix on the pressure space. From the snapshot matrices S_u and S_p we compute two POD bases with the method of snapshots,

$$(S_u X S_u^\top)\, v_i = \lambda_i v_i, \qquad V_u = S_u^\top [v_1, \dots, v_N]\, \Lambda^{-1/2}, \tag{4}$$

and similarly V_p with M_p instead of X. The columns of V_u are X-orthonormal and the columns of V_p are M_p-orthonormal; a Cholesky re-orthonormalisation is applied to remove the loss of orthogonality that the method of snapshots suffers when the eigenvalues approach machine precision. Since B is the discrete divergence and the snapshots satisfy the continuity equation, V_u is divergence-free up to rounding errors ($\|BV_u\|/\|V_u\| \approx 10^{-14}$ in our run).

Because Taylor–Hood stability is not inherited automatically by the reduced spaces, the velocity space is enriched with *supremizers*, $V_s = X^{-1}B^\top V_p$ (then X-orthonormalised, which does not change the space), which guarantee that the reduced inf-sup matrix $B_{sp} = V_s^\top B^\top V_p$ is invertible. We use $N = 40$ modes for velocity, pressure and supremizers. Figure 1 shows the decay of the eigenvalues and the mean projection error on the test set, which is the lower bound for any POD-based method: it is $5.7{\cdot}10^{-3}$ (velocity) at $N = 20$, $5.0{\cdot}10^{-4}$ at $N = 30$ and $9.7{\cdot}10^{-5}$ at $N = 40$ (for pressure: $9.8{\cdot}10^{-3}$, $1.0{\cdot}10^{-3}$, $2.2{\cdot}10^{-4}$). All errors in this report are relative errors in the physical norms: $\|\cdot\|_X/\|\cdot\|_X$ for the velocity and $\|\cdot\|_{L^2}/\|\cdot\|_{L^2}$ for the pressure.

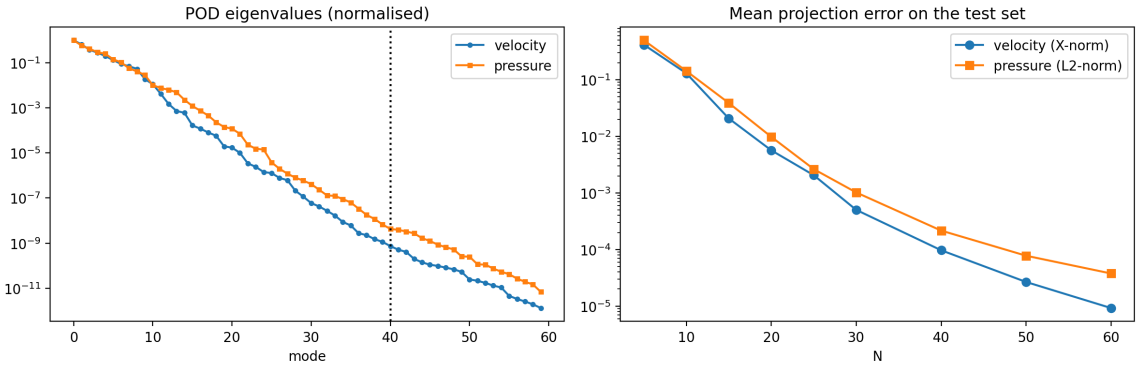


Figure 1

3.2 POD-Galerkin

The velocity is approximated as $\mathbf{u} \approx V_u\mathbf{c} + V_s\mathbf{c}_s$ and the pressure as $p \approx V_p\mathbf{p}_r$. Testing the continuity equation with V_p and using $BV_u \approx 0$ gives $B_{sp}^\top \mathbf{c}_s = \mathbf{0}$, hence $\mathbf{c}_s = \mathbf{0}$: the supremizers

do not contribute to the velocity, they only provide test functions to recover the pressure. The reduced saddle-point system can therefore be condensed *exactly* into two small problems:

$$\text{velocity:} \quad \mu_0\,\mathbf{c} + \tfrac{1}{2}C_r(\mathbf{c})\,\mathbf{c} = \mathbf{f}_u(\mu_1), \tag{5}$$

$$\text{pressure:} \quad B_{sp}\,\mathbf{p}_r = \mu_0\,V_s^\top X V_u\,\mathbf{c} + \tfrac{1}{2}V_s^\top C(V_u\mathbf{c})\,V_u\mathbf{c} - \mathbf{f}_s(\mu_1), \tag{6}$$

where $\mathbf{f}_u = V_u^\top \mathbf{f}$, $\mathbf{f}_s = V_s^\top \mathbf{f}$ and $C_r(\mathbf{c}) = V_u^\top C(V_u\mathbf{c})V_u$. Equation (5) is solved by Newton's method with Jacobian $\mu_0 I + C_r(\mathbf{c})$. The velocity obtained in this way is identical to the one of the full reduced saddle-point system; the condensation only changes how the system is solved (see Section 3.4).

The nonlinearity is quadratic, so $C(\mathbf{u})$ is linear in $\mathbf{u}$ and $N(\mathbf{u}) = \frac{1}{2}C(\mathbf{u})\mathbf{u}$; we verified both properties numerically (relative errors $\sim 10^{-16}$). This is the key for the efficiency of the offline/online split: offline we compute the third-order tensor $T_k = [V_u\ V_s]^\top C(\varphi_k)\,V_u$, $k = 1,\dots,N$ (φ_k the k-th velocity mode), so that $C_r(\mathbf{c}) = \sum_k c_k T_k$ online. The forcing depends on μ_1 only, so its projections $\mathbf{f}_u, \mathbf{f}_s$ are tabulated on a fine grid in μ_1 and interpolated with a cubic spline (interpolation error $\sim 10^{-9}$). The online stage then involves only matrices of size $N \times N$ and its cost is independent of the FOM dimension.

3.3 PODNN

In the PODNN two fully connected networks map $(\log\mu_0, \mu_1)$ to the first $K = 30$ POD coefficients of velocity and pressure,

$$\mathbf{u}(\boldsymbol{\mu}) \approx V_u\,\mathcal{N}_u(\boldsymbol{\mu}), \qquad p(\boldsymbol{\mu}) \approx V_p\,\mathcal{N}_p(\boldsymbol{\mu}). \tag{7}$$

The targets are the coefficients of the training snapshots, $\mathbf{c} = V_u^\top X\mathbf{u}$ and $\mathbf{p}_r = V_p^\top M_p\,p$. The bases are the same as for the Galerkin method. Each network has 3 hidden layers of 64 tanh units; training is described in Section 3.4. Since the bases are orthonormal, $\|\mathbf{u} - V_u\mathbf{a}\|_X^2 = \|\mathbf{c}-\mathbf{a}\|_2^2 + (\text{truncation})^2$: the mean-squared error on the coefficients is exactly the physical error, which makes the loss meaningful.

3.4 Optimisation work

A first "naive" implementation worked but showed three main weaknesses: a badly conditioned reduced system, an online stage that still assembled full-order operators, and a PODNN that was under-trained. Important disclaimer: at first, I've tried some techniques learned in previous classes (e.g. Numerical Optimisation) to reduce the conditioning number, like diagonal shift, Schur, natural scaling etc, but none of them gave a significant result. I've then asked Claude for help and this part is his doing. The proof of the main point of this discussion is listed at the end 5.2.

3.5 Conditioning of the reduced system

In the first version the reduced matrix $J_{lin,r} = \mu_0 A_r - B_r$ of the full saddle-point system had a condition number between 10^2 and 10^6, growing with μ_0. The analysis showed that, since $X = A$ and V_u is divergence-free, the matrix has the block structure

$$J_{lin,r} = \begin{bmatrix} \mu_0 I & 0 & 0 \\ 0 & \mu_0 I & -B_{sp} \\ 0 & -B_{sp}^\top & 0 \end{bmatrix}, \qquad \operatorname{cond}(J_{lin,r}) \sim \frac{\mu_0^2}{\sigma_{\min}(B_{sp})^2}. \tag{8}$$

The pressure basis was normalised in the Euclidean norm of the nodal values, which is mesh-dependent: the entries of B_{sp} were $O(10^{-2})$ and the pressure coefficients $O(10)$, so the ratio between $\mu_0 I$ and B_{sp} was very large. We introduced four changes:

1. the pressure POD is computed in the L^2 norm (M_p), so that B_{sp} is $O(1)$ instead of $O(10^{-2})$;
2. the supremizers are built from the pressure modes and X-orthonormalised: with V_s X-orthonormal and V_p M_p-orthonormal the singular values of B_{sp} lie in $(0, 1]$ (we find $\sigma_{\max} = 1.00$, $\sigma_{\min} = 0.27$, $\mathrm{cond}(B_{sp}) = 3.7$) and $\sigma_{\min}$ is the reduced inf-sup constant;
3. the POD uses a Cholesky re-orthonormalisation (stable for larger N);
4. the system is solved by static condensation, (5)–(6).

Table 1 reports the condition numbers for $N = 40$. The two-by-two blocks $\left(\begin{smallmatrix} \mu_0 & -\sigma \\ -\sigma & 0 \end{smallmatrix}\right)$, one for each singular value σ of B_{sp}, have eigenvalues $(\mu_0 \pm \sqrt{\mu_0^2 + 4\sigma^2})/2$, so $\mathrm{cond}(J_{lin,r}) \approx \mu_0^2/\sigma_{\min}^2$ for $\mu_0 \gg \sigma_{\min}$: this reproduces the table (old basis, $\mu_0 = 10$: $100/(9.5 \cdot 10^{-3})^2 \approx 1.1 \cdot 10^6$; corrected basis: $100/0.27^2 \approx 1.4 \cdot 10^3$). The corrected basis reduces $\mathrm{cond}(J_{lin,r})$ by almost three orders of magnitude at $\mu_0 = 10$; the dependence on μ_0 that remains can be removed by the natural scaling $D = \mathrm{diag}(\mu_0^{-1/2} I, \mu_0^{-1/2} I, \mu_0^{1/2} I)$ (equivalent to $p = \mu_0 \tilde{p}$), which gives a constant value (23.5) for any μ_0 (with the old basis the same scaling leaves $1.1 \cdot 10^4$). A Ruiz diagonal equilibration also brings both bases to $O(10)$, showing that much of the ill-conditioning was a scaling problem. The most important effect is that the systems actually solved online are very well conditioned: the Newton Jacobian $\mu_0 I + C_r$ has cond ≤ 1.4 on all test cases and $\mathrm{cond}(B_{sp}) = 3.7$.

	old basis		corrected basis	
μ_0	$\mathrm{cond}(J_{lin,r})$	+ Ruiz	$\mathrm{cond}(J_{lin,r})$	+ natural scaling
0.1	$1.2{\cdot}10^2$	13.8	10.5	23.5
1	$1.1{\cdot}10^4$	13.8	23.5	23.5
10	$1.1{\cdot}10^6$	13.9	$1.4{\cdot}10^3$	23.5

Table 1: Condition number of the reduced linear saddle-point matrix ($N = 40$). "Old basis": pressure POD in the Euclidean norm and POD of the supremizer snapshots.

3.6 Offline/online efficiency

In the first version every Newton iteration of the ROM assembled the convective operator and the forcing on the full mesh, so the online cost still depended on the FOM size (a speed-up of only about 2×). Exploiting the quadratic structure of the nonlinearity and the one-dimensional dependence of the forcing on μ_1 (Section 3) removed every full-order operation from the online phase: the offline stage takes about 31 s (40 operator assemblies for the tensor and the forcing table), and one online solve about 0.6 ms. No hyper-reduction is needed because the reduction of the nonlinear term is exact.

3.7 Training of the PODNN

We studied the PODNN training on the velocity coefficients ($K = 20$ outputs, 800 training / 200 validation snapshots, errors on the test set including the POD truncation). The main observation is that the networks were *under-fitting*: training and validation errors were close ($\approx 4\%$ and $\approx 5.5\%$ for Adam), so more regularisation or validation-based tuning could not help, and the problem is the optimisation. Table 2 summarises the experiments (single run each).

Configuration (velocity, $K = 20$)	Test error
20k epochs, 3×64	$5.5{\cdot}10^{-2}$
60k epochs, 3×64	5.3–$5.8{\cdot}10^{-2}$
20k epochs, 4×64	$4.9{\cdot}10^{-2}$
20k epochs, 3×128	$4.4{\cdot}10^{-2}$
20k epochs, 3×256 ($\approx 4\times$ training time)	$3.8{\cdot}10^{-2}$
20k epochs + random Fourier features ($\sigma = 1$)	$4.7{\cdot}10^{-2}$
20k epochs + L-BFGS (1500 it.)	$2.65{\cdot}10^{-2}$
20k epochs + L-BFGS (5000 it.)	$1.5{\cdot}10^{-2}$

Table 2: Effect of epochs, network size and optimiser on the PODNN velocity error (mean over the 50 test parameters, one run per configuration).

Tripling the epochs or enlarging the network brings modest and costly improvements, while a second-order fine-tuning with L-BFGS (cheap for such a small, smooth, deterministic regression problem) reduces the error by a factor of 3–4. After L-BFGS a gap between training and validation errors appears, so the validation set becomes useful to select the best L-BFGS checkpoint; the final recipe is therefore: input $(\log \mu_0, \mu_1)$ standardised, a single global scale for the coefficients (the loss then equals the physical error), Adam with cosine schedule for 20k epochs followed by up to 5000 L-BFGS iterations with validation-based checkpointing. The pressure network, whose error was already much closer to the POD limit with Adam alone ($\approx 1.5{\cdot}10^{-2}$ against $1.0{\cdot}10^{-2}$ at $K = 20$), uses the same recipe.

4 Results and comparison

All results refer to the 50 test parameters. Table 3 reports accuracy and cost; Figures **??**–6 show the fields, the errors, and their dependence on the parameters. FOM and POD-Galerkin timings were measured in the same session; the PODNN timing comes from another session, so only the order of magnitude is meaningful.

	velocity error		pressure error		online time
Method	mean	max	mean	max	[ms] (speed-up)
FOM ($\mathbb{P}_2$–$\mathbb{P}_1$)	—	—	—	—	632 (1)
POD-Galerkin ($N = 40$)	$9.7{\cdot}10^{-5}$	$1.4{\cdot}10^{-3}$	$2.6{\cdot}10^{-4}$	$7.5{\cdot}10^{-3}$	0.65 (980)
PODNN ($K = 30$)	$1.3{\cdot}10^{-2}$	$2.4{\cdot}10^{-1}$	$6.2{\cdot}10^{-3}$	$2.1{\cdot}10^{-1}$	0.20 (3096)

Table 3: Relative errors w.r.t. the FOM (physical norms) and cost per query.

Accuracy. The POD-Galerkin error coincides with the POD projection error at the same N ($9.7{\cdot}10^{-5}$ for the velocity at $N = 40$), i.e. the Galerkin projection loses nothing with respect to the best approximation in the reduced space. The PODNN error ($1.3{\cdot}10^{-2}$) is about 25 times above the POD limit ($5.0{\cdot}10^{-4}$ at $K = 30$): the bottleneck is the regression of the coefficients, not the basis. The PODNN mean is dominated by a single test point ($\mu_0 = 0.186$, error 0.24); the median velocity error is $6.0{\cdot}10^{-3}$ and only two test points exceed $5{\cdot}10^{-2}$ (Figure 4). The training set contains only 39 snapshots with $\mu_0 < 0.5$ out of 1000, because μ_0 is sampled uniformly while the solution varies mostly on a logarithmic scale of μ_0 (low viscosity, strongest nonlinearity). The POD-Galerkin ROM is more robust in this region, since it enforces the equations instead of interpolating data.

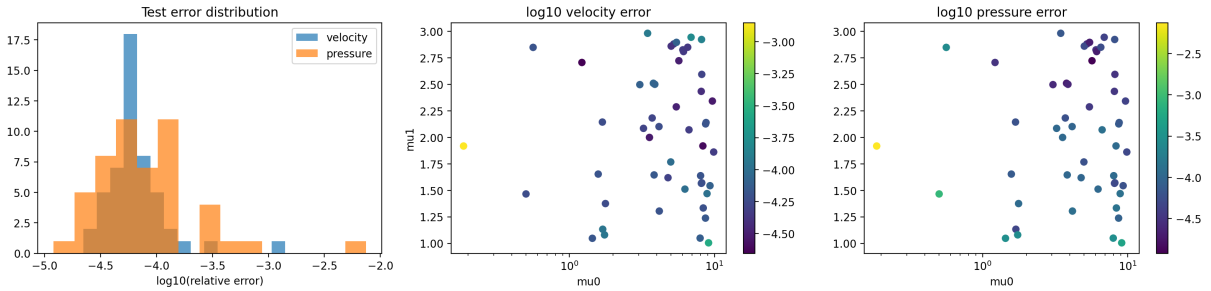


Figure 2: POD-Galerkin on the test set: distribution of the errors (left) and dependence on the parameters (centre: velocity, right: pressure).

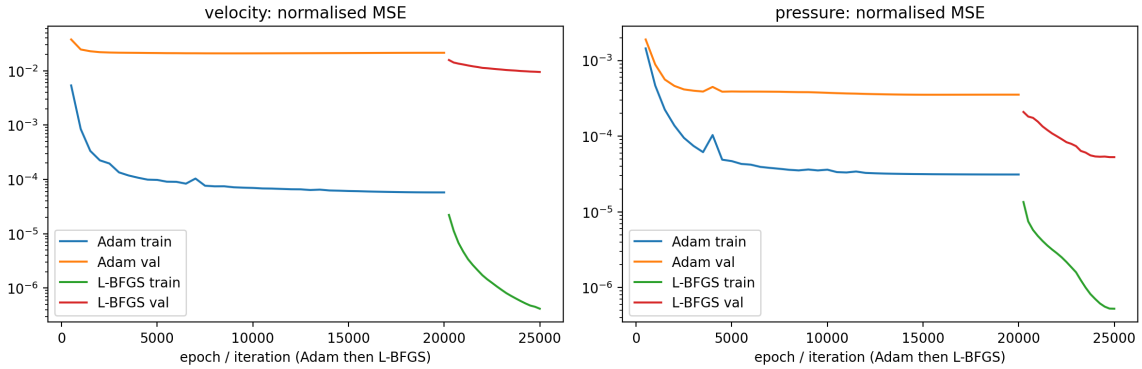


Figure 3: PODNN training histories (normalised MSE): Adam phase followed by L-BFGS iterations, on the training and validation sets.

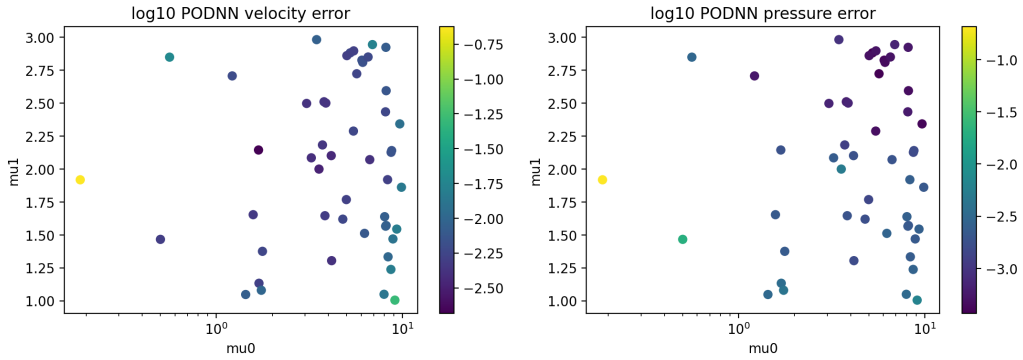


Figure 4: PODNN errors on the test set as a function of the parameters (velocity left, pressure right).

Computational cost. Both ROMs are roughly three orders of magnitude faster than the FOM online, and the difference between them (0.65 vs. 0.20 ms) is irrelevant in practice. The offline costs differ: the POD-Galerkin needs about 31 s (POD, tensor, forcing table), the PODNN about 156 s of training (77 s velocity, 79 s pressure) on top of the same POD. Both share the cost of generating the snapshots (1050 FOM solves, about 10 minutes at 0.6 s each), which dominates the offline phase. The POD-Galerkin ROM is amortised after about $31/0.632 \approx 49$ queries (not counting the snapshots).

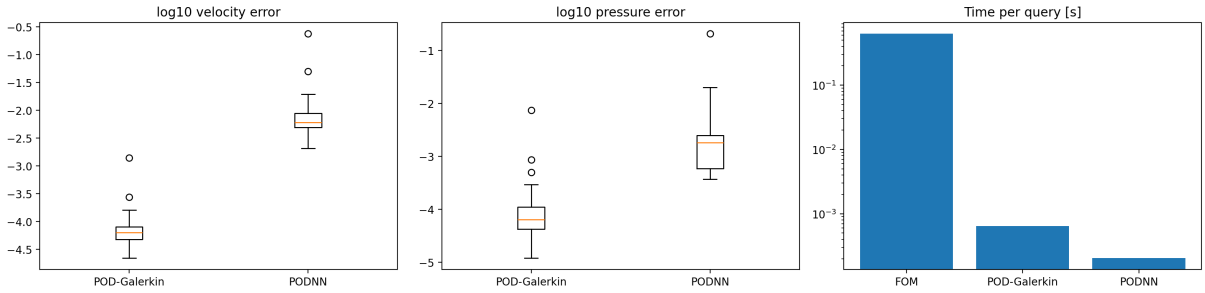


Figure 5: Comparison of the error distributions and of the time per query

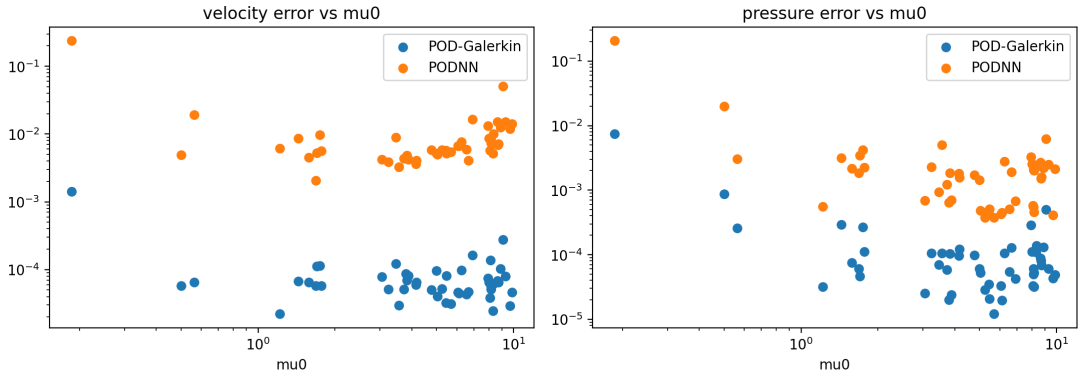


Figure 6: Error of the two methods as a function of μ_0

Comparison: In this problem the POD-Galerkin method dominates the PODNN: it is more accurate by two orders of magnitude, more robust at low μ_0, cheaper to build and comparably fast online (sub-millisecond for both). Its price is that it is intrusive: it needs access to the discrete operators and to the structure of the nonlinearity (here exactly quadratic, which made the offline/online split exact). The PODNN needs only snapshots and could be applied to a solver treated as a black box, and its online cost does not depend on the nonlinearity; however it requires many more training samples than the dimension of the parameter space would suggest, and its accuracy is limited by the regression rather than by the reduced space. For a two-parameter problem with a polynomial nonlinearity the intrusive approach is clearly preferable.

5 PINN

As an optional alternative strategy, we solved the parametrized Navier-Stokes problem using a Physics-Informed Neural Network (PINN). Unlike the POD-Galerkin and PODNN approaches, which rely on a preliminary dataset of finite element snapshots (data-driven), the PINN is a physics-driven, mesh-free method that solves the governing equations by directly minimizing the PDE residuals and boundary conditions within the neural network training loop.

5.1 Architecture and Loss Formulation

The network is parametrized as $\hat{w}(\mathbf{x};\boldsymbol{\mu}) = (u_x, u_y, p)$, mapping the spatial coordinates $\mathbf{x} = (x, y)$ and parameters $\boldsymbol{\mu} = (\mu_0, \mu_1)$ directly to the velocity components and pressure. A fully connected architecture was employed, utilizing tanh activation functions across the hidden layers.N-S equations. The required spatial derivatives of the network output are evaluated as:

$$R_x = -\mu_0 \left(\frac{\partial^2 u_x}{\partial x^2} + \frac{\partial^2 u_x}{\partial y^2} \right) + u_x \frac{\partial u_x}{\partial x} + u_y \frac{\partial u_x}{\partial y} + \frac{\partial p}{\partial x} - f_x \tag{9}$$

$$R_y = -\mu_0 \left(\frac{\partial^2 u_y}{\partial x^2} + \frac{\partial^2 u_y}{\partial y^2} \right) + u_x \frac{\partial u_y}{\partial x} + u_y \frac{\partial u_y}{\partial y} + \frac{\partial p}{\partial y} - f_y \tag{10}$$

$$R_{div} = \frac{\partial u_x}{\partial x} + \frac{\partial u_y}{\partial y} \tag{11}$$

The total loss function $\mathcal{L}_{\text{total}}$ is formulated as a weighted combination of four distinct residual components:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{m}} \mathcal{L}_{\text{m}} + \lambda_{\text{d}} \mathcal{L}_{\text{d}} + \lambda_{\text{bc}} \mathcal{L}_{\text{bc}} + \lambda_p \mathcal{L}_p, \tag{12}$$

where:

- **Momentum Residual:** Measures the violation of the momentum conservation equations evaluated at collocation points distributed across the internal spatial domain Ω.

$$L_{mom} = \frac{1}{N_p} \sum_{k=1}^{N_p} \left[\left(\frac{R_x(\mathbf{x}_p^k, \boldsymbol{\mu}_p^k)}{\mu_{1,k}^2 \pi^2} \right)^2 + \left(\frac{R_y(\mathbf{x}_p^k, \boldsymbol{\mu}_p^k)}{\mu_{1,k}^2 \pi^2} \right)^2 \right] \tag{13}$$

- **Divergence Residual:** Enforces the incompressibility constraint (continuity equation, $\nabla \cdot \mathbf{u} = 0$) over the collocation domain via automatic differentiation.

$$L_{div} = \frac{1}{N_p} \sum_{k=1}^{N_p} R_{div}(\mathbf{x}_p^k, \boldsymbol{\mu}_p^k)^2 \tag{14}$$

- **Boundary Conditions:** Ensures that the network satisfies the prescribed boundary conditions ($\partial\Omega$), such as no-slip walls or inlet/outlet constraints.

$$L_{BC} = \frac{1}{N_b} \sum_{k=1}^{N_b} \left(u_x(\mathbf{x}_b^k, \boldsymbol{\mu}_b^k)^2 + u_y(\mathbf{x}_b^k, \boldsymbol{\mu}_b^k)^2 \right), \quad \text{for } (\mathbf{x}_b^k, \boldsymbol{\mu}_b^k) \in \partial\Omega \times \mathcal{P} \tag{15}$$

- **Pressure:** Accounts for pressure constraints, gauge conditions, or data-fitting terms relative to reference full-order model data across the parameter space $\boldsymbol{\mu}$.

$$L_p = \frac{1}{N_g} \sum_{k=1}^{N_g} p(0, 0, \boldsymbol{\mu}^k)^2 \tag{16}$$

$$\mathcal{L} = \frac{1}{N_b} \sum_{k=1}^{N_b} \left(u_x(\mathbf{x}_b^k, \boldsymbol{\mu}_b^k)^2 + u_y(\mathbf{x}_b^k, \boldsymbol{\mu}_b^k)^2 \right), \quad \text{for } (\mathbf{x}_b^k, \boldsymbol{\mu}_b^k) \in \partial\Omega \times \mathcal{P} \tag{17}$$

5.2 Training Strategy and Optimization

Training PINNs for non-linear systems like Navier-Stokes is notoriously challenging due to stiffness and competing loss gradients. To ensure robust convergence and avoid local optimization plateaus, a structured training strategy is adopted:

- **Exploration Phase (Adam Optimizer):** Initialized using first-order gradient-based optimization coupled with a learning rate scheduler. This phase efficiently drives down boundary errors and rapidly stabilizes the initial physical residuals.
- **Fine-Tuning Phase (L-BFGS):** Transitioning to second-order optimization algorithms (such as L-BFGS with a strong Wolfe line search) initialized from the pre-trained Adam weights. This phase effectively bypasses optimization plateaus, smoothing out stiff gradients and driving the final composite loss down to target tolerances.

Table 4: PINN training convergence history and individual loss components across selected epochs.

Epoch	Total Loss	Momentum	Divergence	Boundary	Pressure	Learning Rate
23000	4.7×10^{-3}	2.0×10^{-3}	1.6×10^{-4}	1.0×10^{-4}	2.3×10^{-5}	5.7×10^{-4}
23250	6.2×10^{-3}	2.1×10^{-3}	2.7×10^{-4}	1.4×10^{-4}	1.2×10^{-6}	5.6×10^{-4}
23500	4.9×10^{-3}	2.2×10^{-3}	1.3×10^{-4}	1.3×10^{-4}	1.8×10^{-5}	5.5×10^{-4}
23750	4.4×10^{-3}	1.9×10^{-3}	1.2×10^{-4}	1.3×10^{-4}	3.7×10^{-5}	5.4×10^{-4}
24000	4.5×10^{-3}	1.9×10^{-3}	1.4×10^{-4}	1.2×10^{-4}	2.9×10^{-5}	5.4×10^{-4}
24250	5.1×10^{-3}	2.1×10^{-3}	2.2×10^{-4}	7.2×10^{-5}	1.5×10^{-5}	5.3×10^{-4}
24500	4.5×10^{-3}	2.0×10^{-3}	1.5×10^{-4}	9.9×10^{-5}	1.3×10^{-5}	5.2×10^{-4}
24750	4.7×10^{-3}	1.9×10^{-3}	1.6×10^{-4}	1.2×10^{-4}	6.7×10^{-5}	5.1×10^{-4}
25000	5.1×10^{-3}	2.1×10^{-3}	1.9×10^{-4}	1.1×10^{-4}	3.9×10^{-5}	5.1×10^{-4}
25250	4.1×10^{-3}	1.9×10^{-3}	1.2×10^{-4}	9.2×10^{-5}	2.5×10^{-6}	5.0×10^{-4}
25500	4.2×10^{-3}	1.9×10^{-3}	1.6×10^{-4}	7.3×10^{-5}	2.9×10^{-6}	4.9×10^{-4}
25750	4.0×10^{-3}	1.8×10^{-3}	1.6×10^{-4}	5.9×10^{-5}	2.0×10^{-6}	4.8×10^{-4}

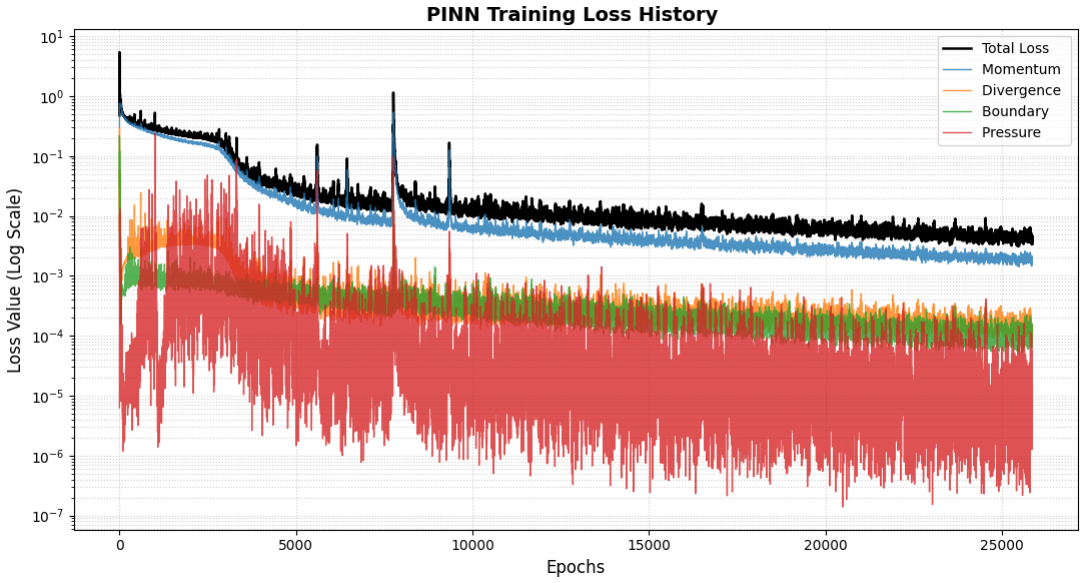


Figure 7

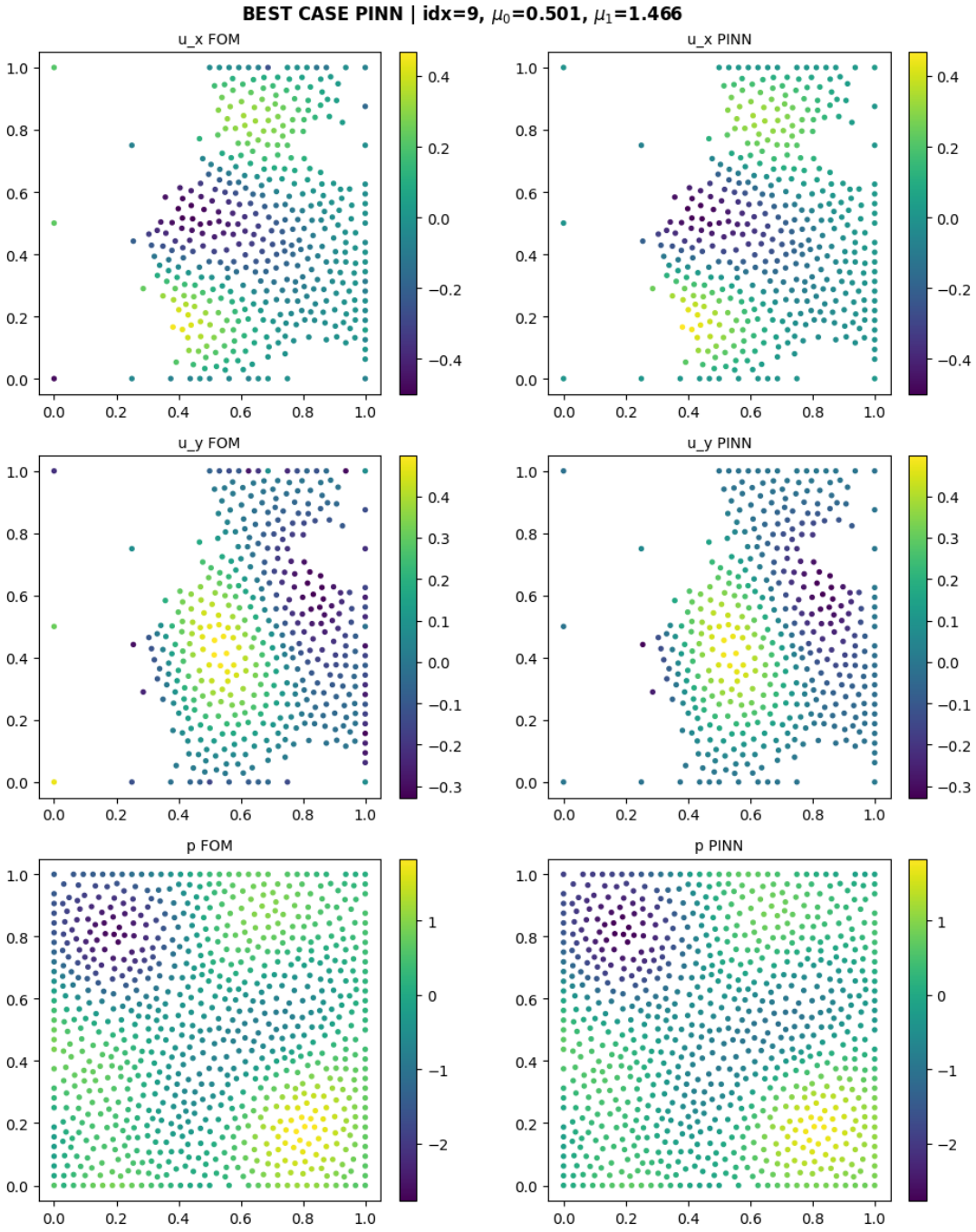


Figure 8: FOM vs PINN's best result. Relativ error: 4.36×10^{-1}

Proposition 1 (exact condition number). *Let $\sigma_{\max} \geq \cdots \geq \sigma_{\min} > 0$ be the singular values of B_{sp}. The singular values of $J_{lin,r}$ are μ_0 (with multiplicity N) and, for each singular value σ of B_{sp}, the two numbers $\frac{1}{2}\big(\sqrt{\mu_0^2 + 4\sigma^2} \pm \mu_0\big)$. Hence*

$$\operatorname{cond}(J_{lin,r}) = \frac{\sqrt{\mu_0^2 + 4\sigma_{\max}^2} + \mu_0}{2\,\min\left\{\mu_0,\ \frac{1}{2}\big(\sqrt{\mu_0^2 + 4\sigma_{\min}^2} - \mu_0\big)\right\}}. \tag{18}$$

Proof. Let $B_{sp} = U\Sigma Z^\top$ be an SVD. The change of basis $\operatorname{diag}(I, U, Z)$ is orthogonal and turns the lower $2N \times 2N$ block of (8) into N independent symmetric blocks $\left(\begin{smallmatrix} \mu_0 & -\sigma_i \\ -\sigma_i & 0 \end{smallmatrix}\right)$. Each one has eigenvalues $\frac{1}{2}(\mu_0 \pm \sqrt{\mu_0^2 + 4\sigma_i^2})$ of opposite sign, and for a symmetric matrix the singular values are the absolute values of the eigenvalues. Both $\frac{1}{2}(\sqrt{\mu_0^2 + 4\sigma^2} \pm \mu_0)$ increase with σ, and the larger one is never smaller than μ_0, so the largest singular value of $J_{lin,r}$ is attained for $\sigma_{\max}$ and the smallest is either μ_0 or the smaller root for $\sigma_{\min}$. □

Two regimes follow from (18): for $\mu_0 \gg \sigma_{\max}$ one has $\operatorname{cond}(J_{lin,r}) \simeq \mu_0^2/\sigma_{\min}^2$, which grows quadratically with the viscosity, whereas for $\mu_0 \ll \sigma_{\min}$ one has $\operatorname{cond}(J_{lin,r}) \simeq \sigma_{\max}/\mu_0$. The expression reproduces the measured condition numbers to all printed digits: with $\sigma_{\max} = 1.000$ and $\sigma_{\min} = 0.2712$ it gives 10.51, 10.78, 23.51, 66.84, 354.1, 1374 for $\mu_0 = 0.1$, 0.5, 1, 2, 5, 10, identical to the values computed directly from $J_{lin,r}$ (Table 1 lists three of them).